

UNIFIED PRIORITY ENGINE SPEC

Autonomous AI Response System for Hospitality Venues

Purpose

This document defines a single priority engine that can rank and respond to medical incidents, fires, natural hazards, infrastructure failures, and crowd-operational crises inside a hospitality venue. The goal is to make the autonomous response system evaluate what is most dangerous right now for the people in the venue, rather than blindly trusting the incident label.

Design Principle

The system should not ask:

- Is fire always more important than flood?
- Is medical always more important than infrastructure?

The system should ask:

- Which incident creates the greatest immediate harm right now, in this venue, for these people, under these access and response constraints?

1. Master Formula

The unified engine uses one master score for all domains:

Final Priority Score = $\max(\text{OverrideFloor}, \min(10, 0.20 * \text{BCP} + 0.80 * \text{LIS} + \text{CompoundModifier}))$

$\text{LIS} = 0.30V + 0.20S + 0.20I + 0.15H + 0.10L + 0.05P$

Where:

- BCP = Base Context Prior, 0-10
- V = Vital / Life Threat, 0-10
- S = Severity / Spread, 0-10
- I = Immediate Intervention Need, 0-10
- H = Historical / Hazard Context, 0-10
- L = Location / Access Risk, 0-10
- P = Propagation / Population Impact, 0-10
- OverrideFloor = deterministic minimum score when certain high-risk conditions are present
- CompoundModifier = 0-1.5 additional risk when overlapping incidents worsen one another

Reasoning

- The outer layer gives 20% weight to a venue-specific baseline prior.
- The live incident score receives 80% weight so current evidence dominates static assumptions.
- This means a coastal hotel in monsoon season has higher flood sensitivity, but a minor drip still will not outrank a live cardiac arrest.

2. Universal Meanings of the Six Factors

- V: How close this event is to immediate loss of life or severe harm.
- S: How severe the event is right now and how fast it is spreading or worsening
- I: How urgently an intervention is needed.
- H: Guest, site, or system context that makes this incident more dangerous.
- L: How difficult it is to reach, contain, rescue, or evacuate.
- P: How many other people may be affected and how much cascade or panic is possible.

3. Priority Bands

- 8.0-10.0 = Critical | Red | Immediate protective action, responders, WebRTC/mesh activation
- 6.0-7.9 = High | Orange | Internal dispatch, external prep, active reranking
- 4.0-5.9 = Medium | Yellow | On-site handling, monitoring, signal collection

- Below 4.0 = Low | Green | Local handling, self-help where appropriate, logging

4. Base Context Prior (BCP)

BCP is the slow-moving context score. It is not the live severity score.

Medical BCP inputs:

- age and frailty
- chronic disease risk
- severe allergy history
- pregnancy or mobility limitations
- event context such as poolside, alcohol-heavy event, isolated guest room

Hazard BCP inputs:

- coastal, hillside, floodplain, or seismic-zone exposure
- monsoon, cyclone, heat, or landslide season
- building height and layout
- occupancy density
- known infrastructure fragility
- prior incidents at this property

Infrastructure BCP inputs:

- building age
- redundancy of power, networking, access control, and vertical transport
- degree of BMS or cloud dependence
- event-load and occupancy conditions
- prior outages and maintenance history

5. Domain Adapters

The six-factor formula stays the same across all incidents. Only the sub-score formulas change by domain.

5A. Medical Adapter

Use the medical triage logic directly.

$$\begin{aligned} V_{\text{med}} &= \min(10, R + B + C + U) \\ S_{\text{med}} &= \min(10, Ps + Ds + As + Ts) \\ I_{\text{med}} &= \min(10, Li + Ri + Ci) \\ H_{\text{med}} &= \min(10, Bc + Cm + Mr + Af) \\ L_{\text{med}} &= \min(10, Al + Hl + Rl) \\ P_{\text{med}} &= \min(10, Np + Cp + Ep) \end{aligned}$$

Medical V sub-scores:

- R = respiration abnormality, 0-4
- B = blood or oxygen abnormality, 0-3
- C = circulation or heart abnormality, 0-2
- U = consciousness state, 0-3

Medical S sub-scores:

- Ps = pain score, 0-4
- Ds = distress behavior, 0-2
- As = acute red-flag symptoms, 0-3
- Ts = time sensitivity, 0-1

Medical I sub-scores:

- Li = lifesaving procedures needed, 0-6
- Ri = risk if delayed, 0-3
- Ci = complexity of intervention, 0-2

Medical H sub-scores:

- Bc = baseline chronic disease risk, 0-4
- Cm = condition-to-event match, 0-3
- Mr = medication risk, 0-2

- Af = age or frailty modifier, 0-1

Medical examples:

- heart attack
- anaphylaxis
- seizure
- stroke
- severe bleeding
- foodborne illness cluster

5B. Fire / Natural Hazard Adapter

Use the venue hazard logic directly.

```
V_haz = min(10, A + E + C + O)
S_haz = min(10, Z + M + D + T)
I_haz = min(10, Li + Ri + Ci)
H_haz = min(10, Bh + Ch + Dh + W)
L_haz = min(10, Al + Hl + Rl)
P_haz = min(10, Np + Cp + Ep)
```

Hazard V sub-scores:

- A = active hazard intensity, 0-4
- E = escalation speed, 0-3
- C = critical systems compromise, 0-2
- O = occupants in immediate danger, 0-3

Hazard S sub-scores:

- Z = zone spread, 0-4
- M = magnitude from sensors or feeds, 0-2
- D = damage signals, 0-3
- T = trend, 0-1

Hazard H sub-scores:

- Bh = baseline site exposure, 0-4
- Ch = condition-to-site match, 0-3
- Dh = dependency fragility, 0-2
- W = warning or compound modifier, 0-1

Hazard examples:

- fire
- flood
- landslide
- earthquake
- gas leak
- smoke spread
- structural damage

5C. Infrastructure / Crowd Adapter

This adapter unifies infrastructure crises and crowd-operational incidents.

```
V_inf = min(10, Ls + Ve + Ex)
S_inf = min(10, Co + Sc + Dm)
I_inf = min(10, Ac + Ri + Rc)
H_inf = min(10, Fg + Dep + Wn)
L_inf = min(10, Eg + Ad + Rt)
P_inf = min(10, Np + Cp + Ep)
```

Infrastructure and crowd sub-scores:

- Ls = life-safety or legality exposure, 0-4
- Ve = velocity of escalation, 0-3
- Ex = exposed or trapped population, 0-3
- Co = continuity disruption, 0-4

- Sc = scope of failure, 0-3
- Dm = damage or functional loss, 0-3
- Ac = action needed now, 0-4
- Ri = risk if delayed, 0-3
- Rc = response complexity, 0-3
- Fg = fragility or compliance gap, 0-4
- Dep = dependency concentration, 0-3
- Wn = warning, peak load, or known weakness, 0-3
- Eg = egress impairment, 0-4
- Ad = access difficulty, 0-3
- Rt = responder delay or blockage, 0-3
- Np = people affected, 0-4
- Cp = cascade risk, 0-3
- Ep = panic or disorder potential, 0-3

Infrastructure and crowd examples:

- elevator entrapment
- keyless lockout
- cyber-physical BMS attack
- internal comms failure
- blackout
- sewage or biohazard failure
- total HVAC collapse
- crowd surge
- capacity breach
- utility emergency
- perimeter breach

6. Legacy Infrastructure Formula

The legacy infrastructure formula remains useful as a quick ops diagnostic but not as the final ranking score.

$$\text{Infra Legacy Score} = ((L * 3) + (C * 2) + V) / 6$$

Where:

- L = life safety and legality
- C = continuity of operations
- V = velocity of escalation

Usage:

- quick internal infrastructure dashboards
- calibration and sanity-checking
- historical incident review

The final autonomous ranking should still use the unified six-factor engine so all incidents compete on the same scale.

7. Hard Override Engine

Certain conditions must impose a deterministic minimum score regardless of the weighted formula.

Medical override examples:

- no pulse or no breathing -> 10.0
- airway compromise or severe anaphylaxis -> 9.5
- major uncontrolled bleeding -> 9.0
- stroke red flags with acute onset -> 8.5
- seizure longer than 5 minutes or repeated seizure -> 9.0

Hazard override examples:

- smoke in an occupied escape corridor -> 8.5
- water above knee level and rising in occupied zone -> 8.5
- trapped occupants -> 9.0

- structural collapse or active landslide impact -> 9.5
- confirmed gas leak with symptoms or ignition risk -> 9.0

Infrastructure and crowd override examples:

- crowd crush indicators at exit -> 9.5
- all egress blocked in occupied zone -> 9.5
- full keyless lockout during evacuation or fire -> 9.0
- cyber-physical control of alarms or BMS during occupancy -> 9.0
- multi-elevator entrapment with medical distress -> 8.5

8. Compound Incident Logic

A single incident may be manageable, but combinations can become critical. Use a compound modifier to reflect this.

CompoundModifier = 0 to 1.5

Recommended rules:

- +0.5 if a secondary incident affects the same zone
- +0.5 if comms or power failure impairs response to the primary incident
- +0.5 if egress or responder routes are degraded by a secondary incident

Examples:

- fire plus internal comms failure
- flood plus basement power outage
- crowd surge plus capacity breach
- guest collapse plus elevator lockout
- gas leak plus smart-building control failure

9. Confidence Score

Priority and confidence should be kept separate.

Confidence = $0.35 * \text{SourceAgreement} + 0.25 * \text{SensorQuality} + 0.20 * \text{Recency} + 0.20 * \text{ModelCertainty}$

Scale:

- 0.0 to 1.0

Interpretation:

- High priority plus high confidence -> auto-execute approved playbook actions
- High priority plus low confidence -> escalate anyway, but mark verify immediately
- Low priority plus low confidence -> gather more signals before escalating

Important rule:

- low confidence must not artificially lower a genuine danger score

10. Autonomy Policy

The AI should not improvise life-safety actions. It should choose from approved playbooks.

Autonomy levels:

- A0 = log only
- A1 = notify staff
- A2 = dispatch internal responders
- A3 = auto-protective actions from allowlist
- A4 = external escalation per policy

Examples of safe A3 actions:

- open WebRTC or local mesh incident room
- push floor-plan evacuation alerts
- disable guest elevators
- unlock approved emergency access doors

- isolate incident zone on dashboard
- route nearest responders
- switch to backup comms

Examples that should remain policy-controlled:

- calling police, fire, or ambulance automatically
- full lockdown
- shutting building-wide utilities without approval logic
- mass notifications outside impacted zones

11. Continuous Reranking

The engine must rescore incidents continuously while they are active.

Suggested reranking cadence:

- every 2-5 seconds: cardiac arrest, seizure, fire, gas leak, crowd surge
- every 5-10 seconds: elevator entrapment, blackout, cyber-physical attack
- every 10-30 seconds: flood, landslide, cyclone, HVAC collapse

Each cycle should:

1. ingest new signals
2. recompute factor scores
3. apply overrides
4. apply compound modifier
5. recompute final score
6. rerank all active incidents
7. refresh actions and explanation

12. Engine Flow

1. Ingest signals from sensors, CCTV analytics, guest apps, staff apps, and external alerts.
2. Detect or classify incident type.
3. Choose the correct domain adapter.
4. Compute Base Context Prior.
5. Compute V, S, I, H, L, and P.
6. Compute Live Incident Score.
7. Apply hard overrides.
8. Apply compound modifier.
9. Compute Final Priority Score.
10. Assign tier and autonomy level.
11. Select playbook actions.
12. Dispatch actions.
13. Continue reranking until resolution.

13. Pseudocode

```
def score_incident(incident, context):
    adapter = choose_adapter(incident.type)

    bcp = adapter.base_context_prior(incident, context)
    factors = adapter.compute_factors(incident, context)

    lis = (
        0.30 * factors["V"] +
        0.20 * factors["S"] +
        0.20 * factors["I"] +
        0.15 * factors["H"] +
        0.10 * factors["L"] +
        0.05 * factors["P"]
    )

    override_floor = override_engine(incident, factors, context)
    compound_modifier = compound_engine(incident, context.active_incidents)
```

```

    final_score = min(10, max(override_floor, 0.20 * bcp + 0.80 * lis + compound
_modifier))
    tier = assign_tier(final_score)
    confidence = confidence_engine(incident)

    actions = policy_engine(
        incident_type=incident.type,
        tier=tier,
        score=final_score,
        confidence=confidence,
        override_applied=(override_floor > 0)
    )

    return {
        "score": round(final_score, 2),
        "tier": tier,
        "confidence": round(confidence, 2),
        "bcp": round(bcp, 2),
        "factors": factors,
        "override_floor": override_floor,
        "compound_modifier": round(compound_modifier, 2),
        "actions": actions
    }

```

14. Canonical Output Object

```

{
  "incident_id": "INC-2041",
  "domain": "hazard",
  "type": "fire",
  "subtype": "corridor_smoke_spread",
  "zone": "Floor 12 East Wing",
  "score": 8.5,
  "tier": "Critical",
  "confidence": 0.88,
  "bcp": 8.0,
  "factors": {
    "V": 8,
    "S": 7,
    "I": 8,
    "H": 6,
    "L": 7,
    "P": 6
  },
  "override_floor": 8.5,
  "compound_modifier": 0.5,
  "reasons": [
    "Smoke in occupied escape corridor",
    "One stairwell degraded",
    "85 people in affected stack",
    "Internet unavailable, local mesh active"
  ],
  "actions": [
    "Push red evacuation popup",
    "Route guests to Stairwell A",
    "Disable guest elevators",
    "Dispatch fire team and medic team",
    "Open WebRTC or mesh incident room"
  ],
  "rerank_after_sec": 5
}

```

15. Example Cross-Domain Ranking

- Guest cardiac event in locked room -> Medical -> High to Critical depending on vitals and overrides
- Smoke in occupied corridor -> Hazard -> Critical
- Crowd surge at banquet exit -> Crowd -> Critical
- Multi-elevator entrapment with no medical distress -> Infrastructure -> High
- Keyless lockout with no active hazard -> Infrastructure -> Medium
- Small contained pantry fire -> Hazard -> Medium unless spread or override occurs
- Extreme flood with blocked exits -> Hazard -> Critical

16. Implementation Guidance

Recommended engineering approach:

- use deterministic rules for thresholds, overrides, evacuation logic, and responder escalation
- use AI for explanation, natural-language summaries, ambiguity resolution, and operator support
- continuously rerank incidents rather than treating severity as static
- log both raw signals and scored outputs for auditability
- separate incident priority from confidence
- gate autonomous actions behind venue-specific policy and compliance rules

17. Final Principle

The unified priority engine works because it keeps one shared score across all crisis types while allowing each domain to use its own evidence model underneath. This creates a single operational language for the response system: one score, one tier, one reranking loop, many incident types.